

1. Đặt vấn đề (Problem and scope)

Trong bối cảnh nhu cầu di chuyển đô thị ngày càng tăng cao tại Việt Nam, các hệ thống đặt xe taxi truyền thống đang bộc lộ nhiều hạn chế nghiêm trọng: không đáp ứng được lưu lượng người dùng lớn trong giờ cao điểm, thiếu khả năng định vị GPS và ghép đôi tài xế thời gian thực, quy trình thanh toán thủ công dễ xảy ra sai sót, mức độ bảo mật thấp và khó mở rộng quy mô. Những vấn đề này dẫn đến trải nghiệm người dùng kém, mất đơn hàng, thời gian chờ đợi dài và rủi ro bảo mật cao.

Hệ thống CAB Booking System được thiết kế nhằm giải quyết toàn diện các vấn đề trên bằng cách áp dụng kiến trúc **Microservices – Real-time – Event-driven – AI-enabled – Zero Trust Architecture**. Hệ thống hướng đến việc xây dựng một nền tảng cloud-native, có khả năng mở rộng linh hoạt, độ tin cậy cao, xử lý thời gian thực và tích hợp trí tuệ nhân tạo để tối ưu hóa quy trình ghép đôi tài xế, tính giá động (surge pricing) và dự đoán thời gian đến (ETA).

Phạm vi nghiên cứu bao gồm:

- Ba nhóm người dùng chính: Hành khách (Customer), Tài xế (Driver) và Quản trị viên (Admin).
 - Các chức năng cốt lõi: đặt xe, ghép đôi thông minh bằng AI, theo dõi GPS thời gian thực, thanh toán an toàn, đánh giá chuyến đi và quản trị hệ thống.
 - Triển khai trên môi trường AWS Cloud với Kubernetes, Terraform, đảm bảo high availability, scalability ngang và bảo mật Zero Trust (“Never trust, always verify”).
 - Không nằm trong phạm vi: huấn luyện chi tiết các mô hình AI, tích hợp phần cứng xe hoặc hệ thống logistics đa phương thức.
- ## 2. Thu thập yêu cầu và khảo sát nghiệp vụ (Requirement and survey)

2.1. Yêu cầu chức năng (Functional Requirements) Hệ thống hỗ trợ đầy đủ các luồng nghiệp vụ sau:

- **Hành khách:** đăng ký/đăng nhập, đặt xe, chọn loại xe & giá, theo dõi chuyến đi thời gian thực, thanh toán và đánh giá.
- **Tài xế:** đăng ký KYC, bật/tắt trạng thái online, nhận chuyến, cập nhật GPS, hoàn thành chuyến đi và xem thu nhập.
- **Quản trị viên:** giám sát real-time, quản lý người dùng/tài xế/chuyến đi, cấu hình surge pricing và xem báo cáo phân tích.

2.2. Yêu cầu phi chức năng (Non-Functional Requirements)

- **Scalability:** Hỗ trợ Horizontal Pod Autoscaling (HPA) và multi-region.
- **Real-time:** WebSocket kết hợp Kafka/RabbitMQ với độ trễ dưới 1 giây.

- **High Availability & Fault Tolerance:** Không có Single Point of Failure (SPOF), áp dụng Circuit Breaker, Retry và Saga Pattern.
- **Security:** Zero Trust Architecture (mTLS, JWT + Refresh Token, WAF, RBAC/ABAC).
- **Observability:** ELK Stack, Grafana và Prometheus.
- **Cloud-native:** Docker + Kubernetes + Terraform + GitHub Actions.

2.3. Khảo sát nghiệp vụ

Hệ thống được thiết kế dựa trên mô hình kinh doanh của các nền tảng đặt xe lớn (Uber, Grab, Be) nhưng được nâng cấp theo tiêu chuẩn hiện đại của ngành công nghiệp, phù hợp với đồ án môn học.

3. Phân tích Domain-Driven Design cho hệ thống

1. Xác định các Actors

- Customer (Hành khách)
- Driver (Tài xế)
- Admin / Operator (Quản trị viên)
- External Actors: AI Matching Service, Payment Gateway (PSP), Map Service (Google Maps / OSRM).

2. Xác định Use Cases

Các use case chính được xác định từ sequence diagrams và UI/UX trong tài liệu thiết kế:

- Login & Refresh Token
- Request Ride (Đặt xe end-to-end)
- AI Driver Matching
- Real-time GPS Update
- Surge Pricing Real-time
- ETA Calculation Real-time
- Payment với Saga Pattern
- Ride Tracking & Notification

- Rating & Feedback
- Admin Monitoring & Management

3. Xác định Domain và Subdomain

- **Core Domain:** Ride Management (quản lý và thực hiện chuyến đi).
- **Supporting Subdomains:** Pricing & Surge, Matching & AI, Payment & Ledger, Notification & Real-time, User Management.
- **Generic Subdomain:** Authentication, Logging, Monitoring

4. Xác định Bounded Context

Mỗi Bounded Context tương ứng với một Microservice (theo nguyên tắc Database-per-Service):

- Auth Context
- Customer Context
- Driver Context
- Booking Context
- Ride Context
- Matching Context (AI)
- Pricing Context
- ETA Context
- Payment Context
- Notification Context

5. Context Mapping: Xác định cách các bounded context giao tiếp với nhau

- Customer → Booking: Giao tiếp đồng bộ qua REST/gRPC và API Gateway.
- Booking → Matching → Ride: Giao tiếp bất đồng bộ qua Kafka (event-driven).
- Ride ↔ Pricing/ETA: Query đồng bộ (gRPC) hoặc qua Redis Cache.
- Ride → Payment: Saga choreography-based.

- Tất cả → Notification: Publish event qua Kafka/RabbitMQ.
- Real-time GPS: WebSocket Gateway + Redis Geo.

6. Xác định Domain Model

- Entities: Customer, Driver, Ride, Payment, Rating.
- Value Objects: Location (GPS), Money, ETA, SurgeFactor.
- Domain Events: RideRequested, DriverMatched, RideStarted, RideCompleted, PaymentSucceeded/Failed.

7. Xác định Aggregate Boundaries

- Ride Aggregate (Root: Ride) – bao gồm trạng thái chuyến đi, vị trí, tài xế, khách hàng, giá cước và tình trạng thanh toán.
- Payment Aggregate (Root: Payment).
- Driver Aggregate – vị trí (Redis Geo), trạng thái online/offline.
- Customer Aggregate – hồ sơ, ví, lịch sử chuyến đi.

8. Xác định MicroServices

- Auth Service
- Customer Service
- Driver Service
- Booking Service
- Ride Service
- Matching Service (AI)
- Pricing Service
- ETA Service
- Payment Service
- Notification Service

9. Thiết kế các giao tiếp giữa MicroServices

- Đồng bộ: REST/gRPC qua API Gateway.

- Bất đồng bộ: Kafka / RabbitMQ.
- Real-time: WebSocket + Redis Pub/Sub.
- Service-to-Service: mTLS qua Service Mesh (Istio/Linkerd).

10. Thiết kế Data Ownership

Áp dụng **Database per Service**:

- PostgreSQL: Ride, Booking, Payment.
- Redis: GPS (Geo index), cache, token blacklist.
- MongoDB (tùy chọn): lịch sử chuyến đi và logs.

Mỗi service chỉ được quyền truy cập database của chính mình.

11. Xác định Integration Pattern

- Saga Pattern (choreography-based) cho Payment.
- Outbox Pattern đảm bảo atomicity giữa DB và Event.
- Circuit Breaker, Retry + Exponential Backoff, Bulkhead Isolation.
- API Gateway làm Policy Enforcement Point (PEP) cho Zero Trust.

12. Refine & Validate

Kiến trúc đã được kiểm chứng thông qua các Failure Scenarios & Recovery Strategies trong tài liệu thiết kế (Auth Service down, Payment timeout, AI failure, Kafka lag...). Đánh giá theo ATAM cho thấy hệ thống đáp ứng đầy đủ các thuộc tính chất lượng: scalability (10x booking), availability, security (Zero Trust) và performance (real-time < 1 giây).

Phân tích DDD trên được xây dựng hoàn toàn dựa trên tài liệu thiết kế kiến trúc CAB Booking System, đảm bảo tính nhất quán end-to-end giữa business domain và kiến trúc kỹ thuật.

4. Thiết kế kiến trúc hệ thống (System architecture design - SAD)

4.1 Kiến trúc tổng thể (Overall Architecture)

Hệ thống CAB Booking System được thiết kế theo kiến trúc Microservices kết hợp Event-driven, đảm bảo tính linh hoạt, khả năng mở rộng ngang (horizontal scalability) và loại bỏ Single Point of Failure (SPOF).

Kiến trúc tổng thể bao gồm ba tầng chính:

- Client Layer: Customer App (React/Next.js), Driver App và Admin Dashboard.
- API Gateway Layer: Làm điểm vào duy nhất, thực thi chính sách Zero Trust (JWT validation, rate limiting, WAF).
- Microservices Layer: Các dịch vụ độc lập, giao tiếp qua REST/gRPC (đồng bộ) và Kafka/RabbitMQ (bất đồng bộ).
- Data & Infrastructure Layer: Database-per-Service, Redis (Geo + Cache), Message Broker và ML Platform.

4.2 Kiến trúc Microservices

Mỗi microservice được xây dựng theo nguyên tắc Database-per-Service, Stateless và Async-first. Danh sách các microservice chính:

Microservice	Trách nhiệm chính	Database
Auth Service	Xác thực, JWT, Refresh Token	PostgreSQL + Redis
Customer Service	Quản lý hồ sơ khách hàng	PostgreSQL
Driver Service	Quản lý tài xế, KYC, trạng thái online	PostgreSQL
Booking Service	Tạo và quản lý yêu cầu đặt xe	PostgreSQL
Ride Service	Quản lý chuyến đi, GPS tracking	PostgreSQL + Redis Geo
Matching Service (AI)	Ghép đôi tài xế thông minh	Redis + Feature Store
Pricing Service	Surge pricing, tính giá động	PostgreSQL + Redis

ETA Service	Dự đoán thời gian đến	Redis
Payment Service	Thanh toán, Saga Pattern	PostgreSQL
Notification Service	Push notification, WebSocket	Redis

4.3 Kiến trúc triển khai (Deployment Architecture)

Hệ thống được triển khai theo mô hình cloud-native trên Kubernetes (hoặc Docker Swarm):

- Multi-region Kubernetes Cluster (Region A & B).
- Horizontal Pod Autoscaling (HPA) tự động scale theo CPU/Memory và custom metrics.
- Global Load Balancer + API Gateway tại Edge.
- Infrastructure as Code: Terraform quản lý toàn bộ tài nguyên AWS.
- CI/CD: GitHub Actions → Docker Build → Kubernetes Deploy.

4.4. Kiến trúc Real-time & Event-driven

- Real-time Layer: WebSocket Gateway (Socket.IO) kết nối trực tiếp Customer App và Driver App.
- Event Broker: Kafka / RabbitMQ làm backbone cho tất cả sự kiện hệ thống (RideRequested, DriverMatched, RideStarted...).
- In-memory Store: Redis GeoIndex cho vị trí tài xế (query < 10ms).
- Latency mục tiêu: < 1 giây cho GPS update và notification.

4.5. Kiến trúc AI & Intelligent Services

AI được tách thành tầng riêng biệt (AI/ML Layer) để không ảnh hưởng đến luồng chính:

- AI Driver Matching: Kết hợp Redis Geo (hard constraint) + Feature Store + ML Model (soft scoring).
- Surge Pricing Model: Tính giá động theo cung-cầu, thời gian, khu vực.
- ETA Prediction Model: Sử dụng lịch sử GPS và traffic data.
- ML Platform: Feature Store + Model Training + Model Serving API.

4.6. Kiến trúc bảo mật – Zero Trust Architecture

Toàn bộ hệ thống tuân thủ nguyên tắc “Never trust, always verify”:

- Client & Edge: HTTPS/TLS 1.3, WAF (chống SQLi, XSS, DDoS), Rate Limiting, Device Fingerprinting.
- API Gateway: Policy Enforcement Point (PEP) – JWT/OAuth2, RBAC/ABAC, Request Validation.
- Service-to-Service: mTLS + Service Mesh (Istio/Linkerd).
- IAM: Central Auth Service, JWT ngắn hạn + Refresh Token rotation, Redis blacklist, MFA cho Admin.
- Data Security: Encryption at-rest & in-transit, Secrets Manager (HashiCorp Vault), GDPR/PDPA compliance.
- Audit & Monitoring: ELK + SIEM + Real-time Alert.

4.7. Kiến trúc mở rộng & chịu lỗi (Scalability & Resilience)

- Horizontal Pod Autoscaling (HPA).
- Circuit Breaker, Retry + Exponential Backoff, Graceful Degradation.
- Eventual Consistency & Async Back-pressure (Kafka).
- Multi-region failover.

4.8. Thiết kế Sequence Diagrams (các luồng quan trọng)

Các sequence diagram chính đã được thiết kế chi tiết:

Quy trình đặt xe end-to-end (Request Ride).

Login & Refresh Token.

Real-time GPS Update (Driver → Passenger).

AI Driver Matching.

Payment Failure & Retry + Saga Pattern (choreography-based).

Surge Pricing Real-time.

ETA Calculation Real-time.

4.9. Tóm tắt công nghệ & chi phí triển khai (Technology Stack)

Back-end: Node.js / NestJS + ExpressJS

Front-end: React.js / Next.js + Tailwind CSS

Event-driven: Kafka / RabbitMQ

Real-time: WebSocket + Socket.IO

Security: Zero Trust + mTLS + JWT

Deployment: AWS + Terraform + Kubernetes

Observability: Grafana + Prometheus + ELK

5. Các mẫu thiết kế phần mềm (Design Pattern)

6. Tối ưu kiến trúc giao dịch (Transaction process): với thanh toán trực tuyến sử dụng micro-service architecture (MSA)
7. Ứng dụng AI (AI / AI agent pipeline)
8. Giám sát hệ thống (System monitor)
9. Bảo mật hệ thống và dữ liệu (Information Security, Data security)
10. Cài đặt và triển khai (CI/CD)
11. Tối ưu hóa hệ thống đáp ứng nhu cầu (System scalability)
12. Kết luận